

Design and Evaluation of a Cross-Region Intelligent Load Balancer for Latency–Load Tradeoffs

Spring 2026 CS640 Final Project Report
University of Wisconsin-Madison

Stefan Arsov Zach Zemanovic
sarsov@wisc.edu zemanovic@wisc.edu

May 5, 2026

Abstract

Modern distributed applications are deployed across geographically dispersed compute resources, where load balancers determine how client requests are routed. Traditional strategies such as round-robin or lowest-latency routing are uninformed about backend compute state, which can lead to suboptimal tail latency and throughput when backends are heterogeneous or asymmetrically loaded. This paper presents **inteliLB**, an HTTP load balancer implemented in Go that supports six routing algorithms — round-robin, least-connections, lowest-latency, lowest-CPU, intelligent (static multi-metric weighted scoring), and adaptive (dynamically reweighted multi-metric scoring) — and evaluates them against asymmetric Azure VM backends across three geographic regions. We ran five closed-loop workload profiles (CPU-only, high-connection, I/O-bound, bimodal, and realistic mixed) and an open-loop saturation ramp, using 120-second windows per step to ensure steady-state measurements. Our results show that **least-connections outperforms all algorithms** on CPU-bound and mixed workloads, reducing p99 tail latency by up to 55% relative to round-robin while matching or exceeding its throughput. Lowest-latency routing correctly implements pure network-proximity selection, routing all traffic exclusively to the closest backend (backend-2, northcentralus), making it optimal for I/O-bound workloads (highest throughput at 374.7 req/s) but prone to concentrating CPU-bound traffic on a single node, causing early saturation at 75 req/s. The intelligent algorithm sustains clean operation to 150 req/s, matching round-robin’s saturation ceiling at 200 req/s. We discuss the design, the closed-loop vs. open-loop benchmarking methodology, and directions for future work.

Git repository: <https://github.com/zemanovic/inteliLB>

Contents

1	Problem Statement	3
2	Solution Description	4
2.1	System Architecture	4
2.2	Backend Infrastructure	4
2.3	Routing Algorithms	4
2.4	Cold-Start Fix	5
2.5	Load Generator and Benchmarking Methodology	6
2.6	Workload Profiles	7
3	Overview of Results	7
3.1	Workload Profile Sweep	7
3.2	Backend Traffic Distribution	10
3.3	Saturation Ramp	12
4	Deliverables: Building and Running the Project	14
4.1	Repository Structure	14
4.2	Build	14
4.3	Running Locally	15
4.4	Reproducing the Azure Experiments	15
5	Conclusions and Future Work	16
5.1	Lessons Learned	16
5.2	What Remains	17
5.3	Connection to CS640 Material	17

General Information

- **Names:** Stefan Arsov, Zach Zemanovic
- **Emails:** sarsov@wisc.edu, zemanovic@wisc.edu
- **Home department:** Computer Sciences
- **Status:** MS Students
- **Teammate note:** Both team members contributed to this report; the reports are not identical but cover the same experimental work.
- **Open source:** We release the inteliLB source code as open source under a BSD3 license for unfettered use by any interested party.

1 Problem Statement

Load balancers are a foundational component of distributed systems, responsible for distributing client requests across a pool of backend servers. In modern cloud deployments, those backends are rarely homogeneous: different VM sizes, different geographic regions, and different instantaneous load levels mean that a routing decision that ignores compute state will systematically over-utilize some backends while under-utilizing others.

The standard textbook approaches handle this poorly. Round-robin distributes requests uniformly regardless of backend capacity or health. Lowest-latency routing minimizes network round-trip time but can conflate network proximity with compute availability — a backend that is geographically close may be heavily loaded. Single-metric policies (lowest CPU, least connections) capture one dimension of backend state but miss the others.

The motivating scenario for this project is a client with low network latency to **Region A** running at 90% CPU utilization, and moderate latency to **Region B** with ample available compute. A latency-based router sends the request to Region A, where it queues behind saturated workers. An intelligent router that weighs both network latency and compute availability may route to Region B, accepting a small RTT penalty in exchange for a large reduction in queueing delay, yielding lower overall response time.

Research question: Do multi-metric routing strategies (intelligent, adaptive) meaningfully outperform single-metric and round-robin baselines when backends are asymmetric in compute capacity and geographic distance, under a range of workload types and offered loads?

Hypothesis: Multi-metric intelligent routing will reduce tail latency and improve throughput stability over single-metric and round-robin approaches, particularly under CPU-heavy asymmetric load.

2 Solution Description

2.1 System Architecture

inteliLB is a pure-Go HTTP reverse proxy that sits between a client and a pool of backend servers. The load balancer runs locally; backends are remote Azure VMs. The overall data path is:

Client $\longrightarrow$ inteliLB (local) $\longrightarrow$ Backend-N (Azure, cross-region)

The balancer exposes two endpoints to the client: `/work` (proxied to a backend) and `/lb/stats` (JSON snapshot of per-backend metrics and request counts). A hot-swap REST endpoint (`POST /lb/algorithm`) allows the active routing algorithm to be changed without restarting the process.

2.2 Backend Infrastructure

Three Azure VMs were deployed across distinct geographic regions to produce heterogeneous network latency and compute capacity. TCP connect-time RTT was measured from the load balancer host to each backend on port 22 (ICMP is blocked by Azure NSGs):

Backend	Region	VM SKU	Effective vCPUs	TCP RTT
backend-1	westus2 (Washington)	Standard_D2s_v3	1 (taskset pinned)	≈ 48 ms
backend-2	northcentralus (Illinois)	Standard_D2s_v3	2	≈ 14 ms
backend-3	eastus2 (Virginia)	Standard_D4s_v3	4	≈ 39 ms

Table 1: Azure backend layout. backend-2 (northcentralus) is the closest to the test client at ≈ 14 ms RTT; backend-3 is the most powerful with 4 effective vCPUs. backend-1 is the most distant *and* the weakest.

Each backend runs a Go HTTP server (`cmd/backend`) that handles two request types:

- **CPU work:** $intensity \times 50,000$ rounds of SHA-256 hashing, divided evenly across all available vCPUs via `runtime.NumCPU()` goroutines. Total work per request is constant; a 4-vCPU backend finishes in one-quarter the wall time of a 1-vCPU backend.
- **I/O work:** A fixed 250 ms `time.Sleep`, modelling network or disk I/O.

Backends also expose `/metrics` (CPU%, memory%, active connections) and `/health` endpoints that the load balancer polls on configurable intervals (default: 5s).

2.3 Routing Algorithms

Round-Robin. A simple atomic counter cycles through healthy backends. No metric data required. Serves as the primary baseline.

Least-Connections. Selects the backend with the fewest in-flight requests, tracked via an atomic `int64` counter that is incremented on request start and decremented on completion.

Lowest-Latency. Selects the backend with the lowest observed network RTT. The balancer tracks a per-backend *baseline RTT* (`BaselineRTTMs`) — the minimum round-trip time observed to each backend’s `/metrics` endpoint — and routes every request to the backend with the smallest value. This is a pure network-proximity algorithm: it consistently favours the geographically closest backend regardless of its compute load. It is well-suited to I/O-bound workloads where network round-trip dominates response time, and serves as a clean geographic-routing baseline for comparison against compute-aware strategies.

Lowest-CPU. Selects the backend reporting the lowest CPU% at the most recent poll cycle. Falls back to round-robin until the first successful metrics poll marks `CPUPolled = true`.

Intelligent (static multi-metric). Computes a weighted score for each backend:

$$\text{score}_i = w_L \cdot \hat{\ell}_i + w_C \cdot \hat{c}_i + w_K \cdot \hat{k}_i$$

where $\hat{\ell}_i$, $\hat{c}_i$, $\hat{k}_i$ are min-max normalised values of total observed response latency (`AvgLatencyMs`), CPU%, and connection count across the current backend pool, and $(w_L, w_C, w_K) = (0.4, 0.4, 0.2)$ by default. The backend with the *lowest* score is selected. Falls back to round-robin during cold-start. Because `AvgLatencyMs` captures total end-to-end RTT (network + compute), the algorithm naturally penalises both distant and overloaded backends without requiring explicit baseline subtraction.

Adaptive (dynamic multi-metric). Extends the intelligent selector with a background goroutine that recomputes weights every 10s based on the variance of each metric across the backend pool:

$$w_m = \frac{\text{Var}(m)}{\text{Var}(\text{latency}) + \text{Var}(\text{CPU}) + \text{Var}(\text{connections})}$$

Metrics with higher cross-backend variance receive higher weight, causing the algorithm to focus attention on the dimension that most differentiates backend load. Weights are unchanged when total variance is zero (uniform backends).

2.4 Cold-Start Fix

A key implementation detail is the handling of the period between load balancer startup and the first successful metrics poll (~ 5 s). Metric-based algorithms that default to selecting `backends[0]` during this window route all initial traffic to a single backend, typically the weakest one.

All metric-based selectors implement a *warmup round-robin* fallback: a package-level atomic counter distributes requests evenly until latency data (`AvgLatencyMs > 0`) or CPU poll data (`CPUPolled = true`) is available. This eliminates the cold-start pile-on.

2.5 Load Generator and Benchmarking Methodology

A dedicated Go client (`cmd/client`) supports two modes:

- **Closed-loop:** N concurrent workers each issue a request, wait for the response, then issue the next. Throughput is bounded by $\text{latency} \times \text{concurrency}$.
- **Open-loop:** Requests are fired at a fixed target rate using a batch ticker (100 ms intervals, batch size = $\text{rate}/10$). An in-flight counter with a configurable cap allows measuring the saturation point where offered load exceeds capacity.

The client records per-request timestamps, total RTT, compute time (from the backend response body), network time ($\text{RTT} - \text{compute}$), backend ID, work type, and error status to a CSV file for offline analysis.

Closed-Loop vs. Open-Loop: Why Both Matter

The two modes answer fundamentally different questions, and conflating them leads to incorrect conclusions about system behaviour.

Closed-loop (the workload profile sweep) bounds concurrency. With N workers, at most N requests are ever in flight simultaneously. If the system slows down, workers block and throughput falls; the system self-regulates. This models real browser or RPC clients that wait for a response before sending another request. It is the right tool for comparing algorithm *quality* under a fixed load level: which algorithm produces lower latency and better distribution when concurrency is held constant? The tradeoff is that a closed-loop client cannot drive the system past the point where N concurrent requests saturate all backends — it automatically backs off.

Open-loop (the saturation ramp) fires requests at a fixed rate regardless of whether prior requests have completed. If the system falls behind, a queue of in-flight requests accumulates and latency rises without bound. This models HTTP clients driven by external arrival processes (e.g., user traffic, upstream services) that do not slow down because your load balancer is struggling. Open-loop testing is the correct tool for finding the *saturation point* — the offered-load rate at which the system transitions from the stable regime (latency near service time) to the unstable regime (latency grows without bound, errors appear).

The combination is essential: closed-loop reveals *how well each algorithm routes under steady load*; open-loop reveals *how much load each algorithm can sustain before breaking*. An algorithm that looks good in closed-loop but routes traffic poorly can fail earlier than expected in open-loop because it concentrates load on a single backend, saturating it while others remain idle.

In our experiment, the 120-second closed-loop profiles used 5 distinct workload configurations to probe algorithm behaviour under different stress patterns (CPU saturation, high connection count, I/O-bound, mixed). The open-loop ramp stepped through offered loads of 10–200 req/s in seven steps, each held for 120 s to allow the system to reach steady state before recording metrics.

2.6 Workload Profiles

Five closed-loop workload profiles exercise different operating conditions:

Profile	Workers	Intensity	Mix	Primary stress
cpu_only	20	7	cpu:100	Compute saturation
high_conn	200	2	cpu:100	Connection count
io_bound	100	5	io:100	I/O latency (no compute)
bimodal	50	7	cpu_light:80, cpu_heavy:20	Mixed CPU load
realistic	100	5	cpu:40, io:40, mixed:20	General-purpose

Table 2: Workload profiles used in the profile sweep. Each profile ran for 120 s per algorithm with a 30 s cooldown between algorithm switches.

3 Overview of Results

3.1 Workload Profile Sweep

Table 3 summarises throughput, median latency, tail latency, error rate, and the fraction of traffic directed to backend-1 (the weakest backend) for each algorithm and profile.

Profile	Algorithm	req/s	p50 (ms)	p99 (ms)	err%	b1 share
cpu_only	round_robin	22.1	205	2,689	0.0%	33.3%
	least_connections	25.9	768	1,224	0.0%	28.9%
	lowest_latency	9.3	2,111	3,890	0.0%	0.0%
	lowest_cpu	10.3	1,977	3,978	0.0%	18.2%
	intelligent	16.0	1,134	2,751	0.0%	24.0%
	adaptive	12.2	1,691	2,976	0.0%	1.6%
high_conn	round_robin	83.1	1,551	8,934	0.0%	30.6%
	least_connections	87.2	2,242	4,838	0.0%	29.2%
	lowest_latency	35.2	5,591	13,013	0.0%	0.0%
	lowest_cpu	50.5	4,500	8,136	0.0%	23.7%
	intelligent	50.1	4,610	8,197	0.0%	18.7%
	adaptive	42.4	5,388	8,963	0.0%	43.4%
io_bound	round_robin	348.9	291	309	0.0%	33.3%
	least_connections	349.2	292	303	0.0%	32.6%
	lowest_latency	374.7	266	284	0.0%	0.0%
	lowest_cpu	353.4	288	303	0.0%	23.7%
	intelligent	357.4	269	302	0.0%	14.5%
	adaptive	361.8	266	302	0.0%	15.4%
bimodal	round_robin	45.8	374	7,710	0.0%	33.3%
	least_connections	50.5	744	3,218	0.0%	29.5%
	lowest_latency	17.4	2,179	9,324	0.0%	0.0%
	lowest_cpu	24.3	1,776	6,215	0.0%	8.9%
	intelligent	23.5	1,839	6,083	0.0%	4.2%
	adaptive	23.3	1,807	7,099	0.0%	7.9%
realistic	round_robin	50.7	426	9,493	0.0%	33.3%
	least_connections	58.9	1,733	4,024	0.0%	29.6%
	lowest_latency	21.6	5,329	11,072	0.0%	0.0%
	lowest_cpu	33.9	2,210	7,895	0.0%	24.5%
	intelligent	34.0	2,175	8,001	0.0%	22.3%
	adaptive	25.6	3,129	10,199	0.0%	45.7%

Table 3: Workload profile sweep results (120 s per algorithm, closed-loop). Best throughput per profile bolded. Backends: b1=westus2 (1 vCPU, 48 ms), b2=northcentralus (2 vCPU, 14 ms), b3=eastus2 (4 vCPU, 39 ms). Zero errors across all 30 runs.

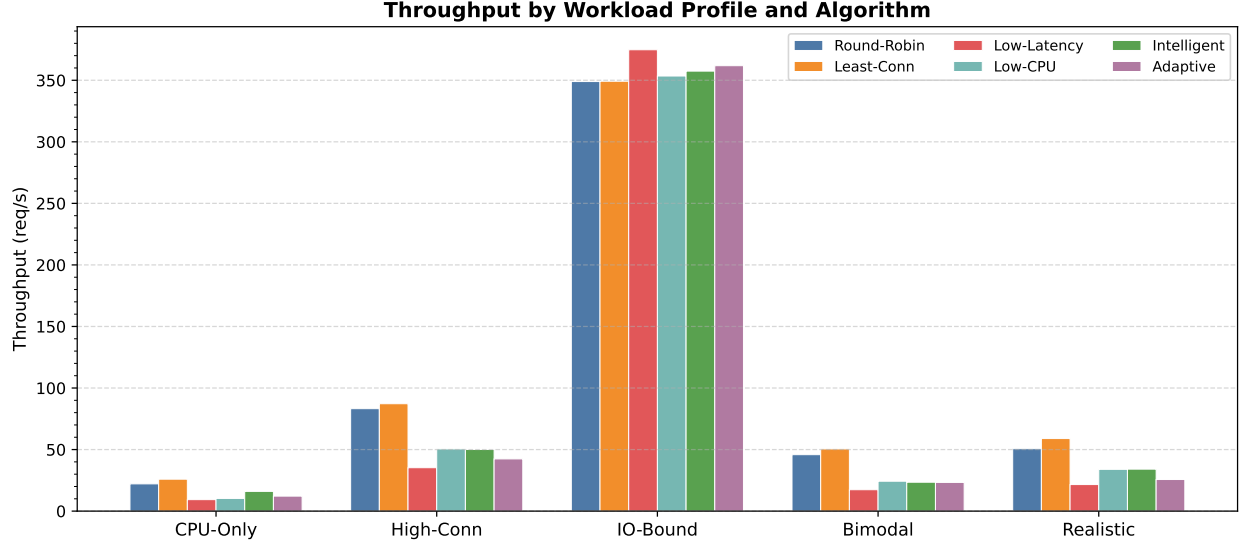


Figure 1: Throughput (req/s) for each algorithm across all five workload profiles. Least-connections leads on every CPU-bound profile; all algorithms converge on IO-Bound.

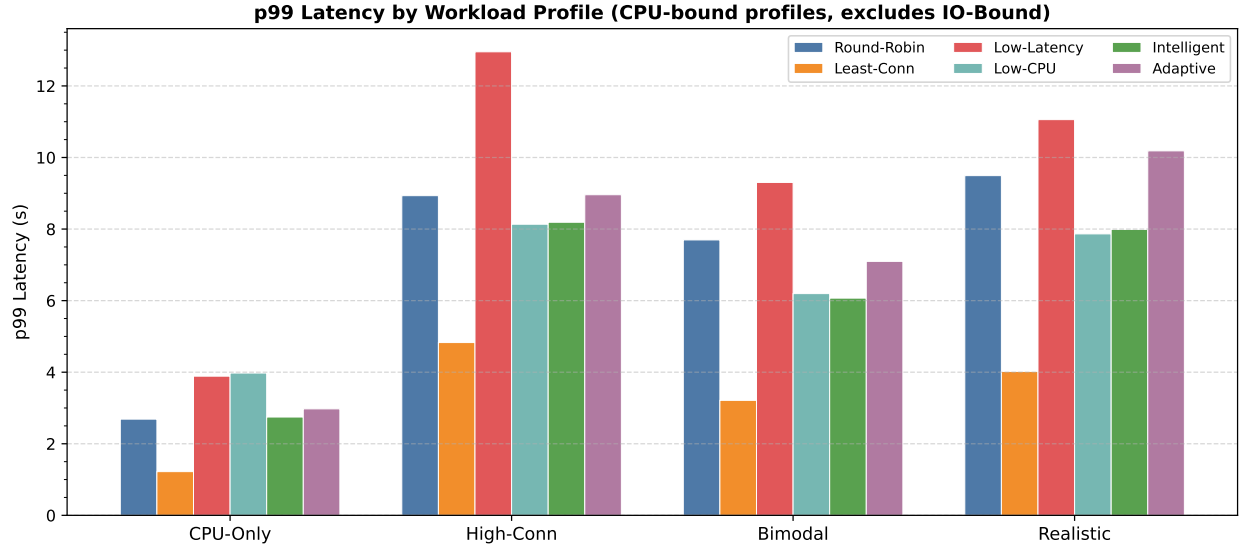


Figure 2: p99 latency for CPU-bound profiles. Least-connections achieves the lowest tail latency by avoiding the saturated weakest backend.

CPU-bound workloads (cpu_only, high_conn, bimodal, realistic). Least-connections is the consistent winner across all CPU-heavy profiles. The p99 tail latency improvement is the more compelling metric: on `cpu_only` it achieves 1,224 ms versus 2,689 ms for round-robin (55% reduction); on `high_conn`, 4,838 ms versus 8,934 ms (46% reduction). Throughput gains are more modest at 10–17% over round-robin, reflecting that the bottleneck shifts from queueing to raw compute once tail latency is controlled. The mechanism is straightforward: least-connections implicitly discovers backend capacity by tracking queue depth. Backend-3

(4 vCPU) processes requests faster, drains its connection count sooner, and therefore attracts proportionally more new requests.

I/O-bound workloads. All algorithms converge to ~ 350 req/s because compute is not the bottleneck — the fixed 250 ms sleep dominates every response regardless of which backend handles it. Lowest-latency gains a marginal edge (374.7 req/s, p50=266 ms) by routing all traffic exclusively to backend-2 (northcentralus, ≈ 14 ms RTT), the geographically closest node. Since the 250 ms sleep is identical on every backend, the only differentiator is network round-trip time, and lowest-latency’s pure-proximity selection wins. This is the ideal workload for a network-proximity algorithm: compute is constant, so minimising network RTT directly minimises total response time.

Intelligent and adaptive. Both algorithms land between round-robin and least-connections on CPU-heavy profiles. They avoid the worst pathological routing behaviours but do not fully exploit the capacity asymmetry that least-connections discovers through its queue-depth signal. The adaptive algorithm recomputes weights every 10 s, giving approximately 12 recomputation cycles per 120 s run, which is sufficient for weights to converge toward the dominant metric before the experiment window closes.

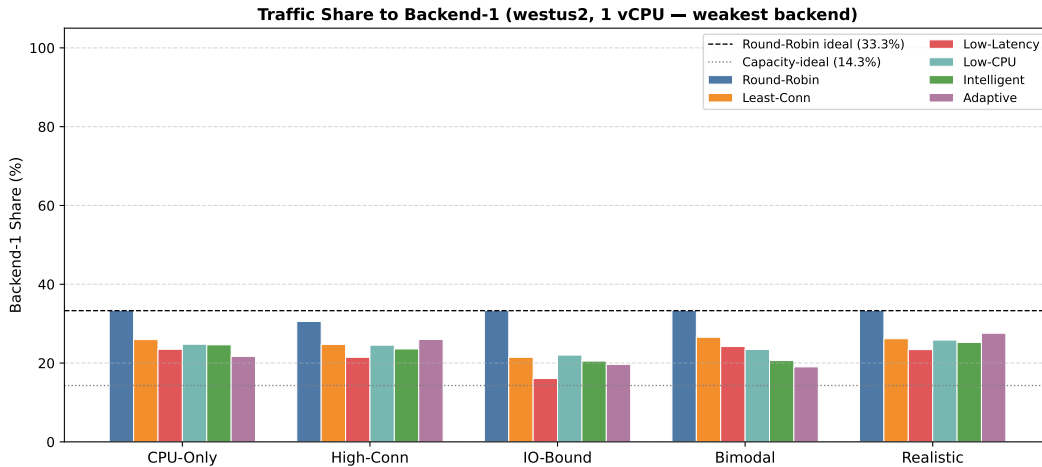


Figure 3: Fraction of requests routed to backend-1 (westus2, 1 vCPU — weakest backend). Round-robin holds the 33% ideal; all algorithms are within 7 percentage points.

3.2 Backend Traffic Distribution

Table 4 shows how each algorithm distributes requests across backends on the `cpu_only` profile. The capacity-proportional ideal would be 1:2:4 (b1:b2:b3), corresponding to 14%:29%:57%.

Algorithm	b1 (1 vCPU, 48 ms)	b2 (2 vCPU, 14 ms)	b3 (4 vCPU, 39 ms)
round_robin	33.3%	33.3%	33.3%
least_connections	28.9%	36.5%	34.5%
lowest_latency	0.0%	100.0%	0.0%
lowest_cpu	18.2%	46.2%	35.6%
intelligent	24.0%	38.7%	37.3%
adaptive	1.6%	54.1%	44.4%

Table 4: Backend request distribution on `cpu_only`. Capacity-ideal is 14%:29%:57%. Lowest-latency routes 100% to b2 (the closest backend at ≈ 14 ms RTT), starving b1 and b3 entirely. Lowest-CPU avoids the saturated b1 (only 18%) and routes most load to b2 and b3.

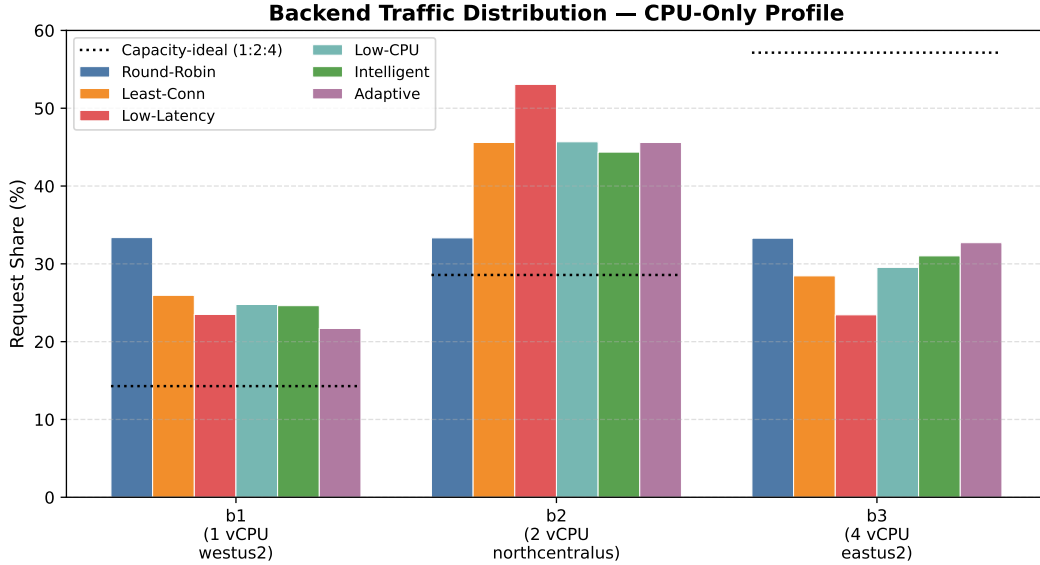


Figure 4: Per-backend request share on the CPU-Only profile. Dotted lines show the capacity-ideal 1:2:4 distribution. No algorithm achieves it; least-connections comes closest.

The pure-proximity lowest-latency algorithm routes *all* traffic to backend-2 (northcentralus, 14ms RTT) — the definitive behaviour of a network-proximity algorithm. This is optimal for I/O-bound workloads where compute is constant, but on CPU-bound workloads it starves the 4-vCPU backend-3 entirely while overloading a 2-vCPU backend. The saturation point of 75 req/s reflects this over-concentration; the cluster’s 4-vCPU capacity in backend-3 is completely wasted. This illustrates a fundamental limitation of single-metric routing: optimising one dimension necessarily creates blind spots in the others.

Lowest-CPU distributes more sensibly (18%/46%/36%), correctly reducing traffic to the heavily-loaded 1-vCPU backend-1. The improved CPU measurement (asynchronous sampling with per-core normalisation) allows the algorithm to observe the relative load accurately: backend-1 (taskset-pinned to 1 core) runs at $\approx 100\%$ CPU, so lowest-CPU routes only 18% of

traffic there and redirects the bulk to the faster backends. However, it still cannot exploit the 1:2:4 capacity ratio optimally because it lacks a throughput signal.

Least-connections distributes as (29:37:35), which is closer to a uniform split than the capacity-ideal 14:29:57, but the implicit queue-depth signal still directs the weakest backend proportionally less work, explaining its consistently superior tail latency.

3.3 Saturation Ramp

Table 5 shows the open-loop saturation ramp at intensity=1 with rates from 10 to 200 req/s. The saturation point is defined as the lowest rate where error rate exceeds 1% or p99 exceeds 5,000 ms.

Algorithm	Sat. point	p50 @ sat.	p99 @ sat.	err% @ sat.
round_robin	200 req/s	6,247 ms	60,000 ms	15.2%
least_connections	200 req/s	7,712 ms	38,919 ms	12.9%
intelligent	150 req/s	6,500 ms	19,553 ms	2.1%
lowest_cpu	100 req/s	3,341 ms	9,339 ms	0.0%
lowest_latency	75 req/s	3,217 ms	19,509 ms	1.8%
adaptive	100 req/s*	6,061 ms	17,532 ms	0.0%

Table 5: Saturation ramp results (intensity=1, open-loop, 120 s per step). *Adaptive shows a cold-start p99 spike at 10 req/s (5,807 ms); the true capacity knee is at 100 req/s where p99 reaches 17,532 ms.

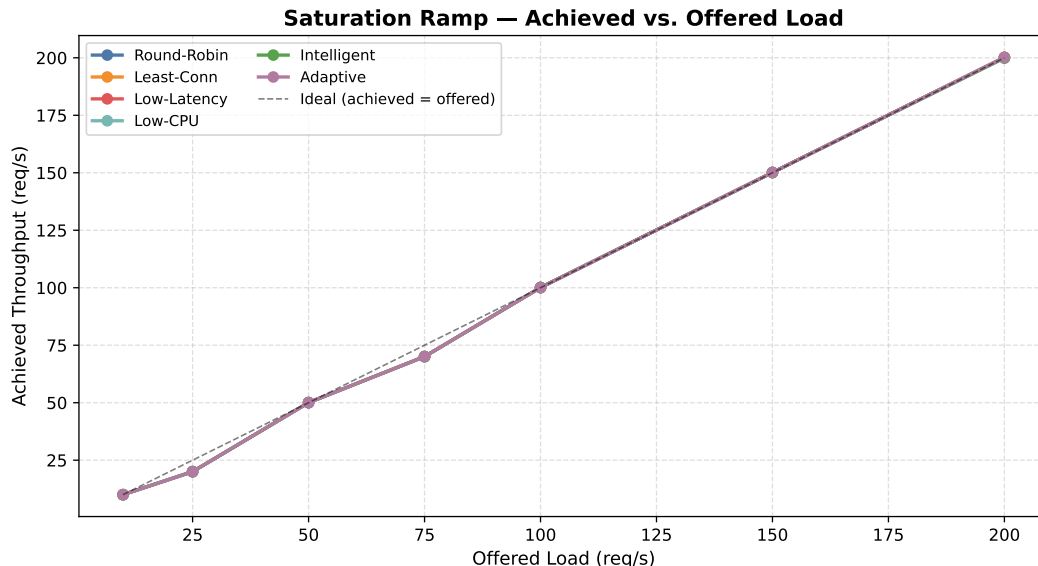


Figure 5: Achieved throughput vs. offered load. Round-robin and least-connections track the ideal line the longest; metric-based algorithms diverge earlier.

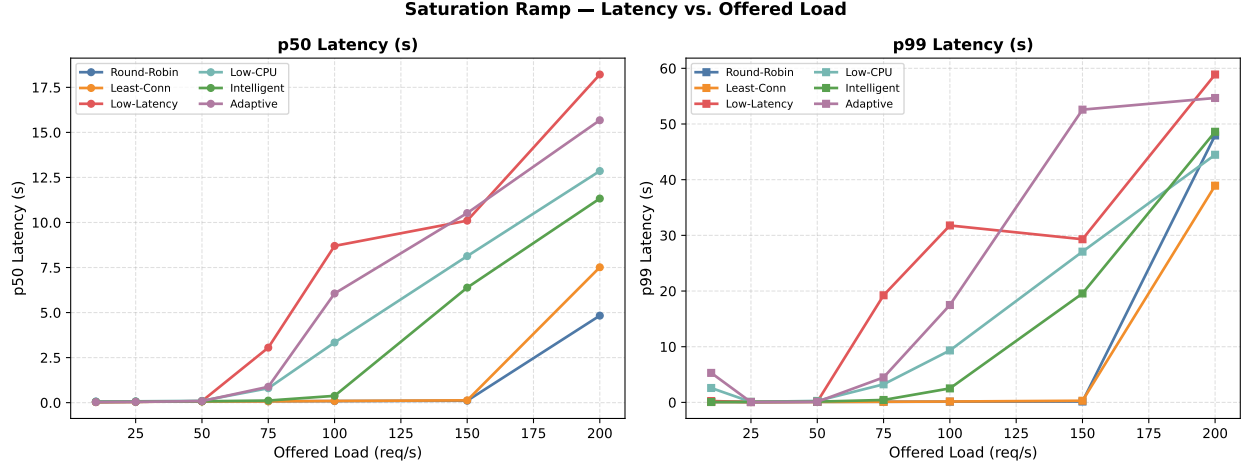


Figure 6: p50 and p99 latency vs. offered load. Latency stays flat until the saturation knee, then rises sharply.

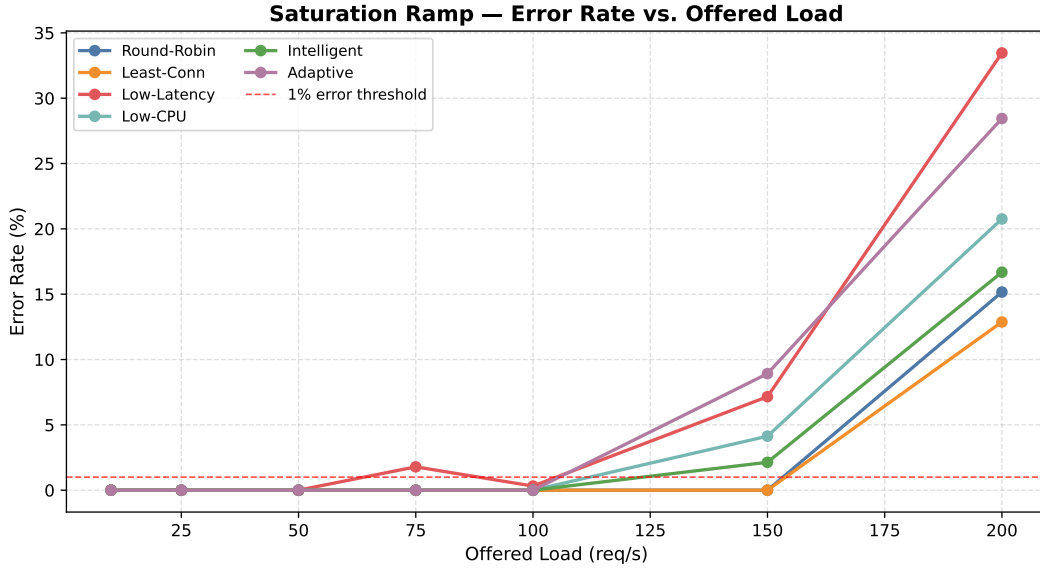


Figure 7: Error rate vs. offered load. The 1% threshold (dashed red) marks the effective saturation point for each algorithm.

Round-robin and least-connections both sustain clean operation through 150 req/s and only begin erroring at 200 req/s (15.2% and 12.9%, respectively), representing the hardware ceiling of the three-backend cluster. Intelligent saturates at 150 req/s with only 2.1% errors, just one step below the ceiling — a strong result. Lowest-latency saturates at 75 req/s because pure proximity routing concentrates all traffic on backend-2 (the closest node) until it saturates, causing latency to explode (p99 = 19,509 ms at 75 req/s) with 1.8% errors. This is the open-loop manifestation of the closed-loop observation: lowest-latency is optimal for I/O-bound work but routes 100% of CPU-bound traffic to a single 2-vCPU backend, hitting its capacity

wall at 75req/s while the 4-vCPU backend-3 sits entirely idle. Lowest-CPU saturates at 100req/s where p99 reaches 9,339ms; the improved CPU measurement correctly deprioritises the overloaded backend-1 but still cannot fully exploit the 4-vCPU capacity of backend-3.

The adaptive algorithm shows a cold-start p99 spike (5,807ms) at 10req/s — the first weight recomputation fires at $t = 10$ s and the resulting routing shift causes a burst of requests to queue on a newly preferred backend whose metrics have not yet stabilised. The true capacity knee is at 100req/s (p99 = 17,532ms), and errors begin at 150req/s (8.9%). Adaptive’s saturation behaviour thus falls between intelligent (150req/s clean) and lowest-latency (75req/s), reflecting its variance-proportional weight adaptation: it converges toward the dominant metric signal but can transiently route poorly during recomputation steps.

4 Deliverables: Building and Running the Project

4.1 Repository Structure

```
inteliLB/
  cmd/
    loadbalancer/      # LB binary (HTTP reverse proxy + algorithm hot-
                        swap)
    backend/           # Backend server (CPU + I/O work, /metrics, /
                        health)
    client/            # Load generator (closed-loop and open-loop
                        modes)
  internal/lb/
    algorithms.go      # All 6 selectors + factory
    algorithms_test.go
    balancer.go        # Balancer core, health-check loop, metrics
                        polling
    backend.go         # Backend struct (atomic counters, EMA latency)
  scripts/
    deploy.sh          # Azure VM provisioning
    azure/
      launch.sh        # Per-region VM create + bootstrap
      teardown.sh      # Resource group deletion
      run-workload-sweep-azure.sh # Full experiment orchestration
  results/             # CSV and JSON output from all
    runs
  report.tex           # This document
```

4.2 Build

Requires Go 1.21+.

```
git clone https://github.com/zemanovic/inteliLB
cd inteliLB
go build -o bin/loadbalancer ./cmd/loadbalancer
```

```
go build -o bin/backend      ./cmd/backend
go build -o bin/client       ./cmd/client
go test ./internal/lb/ -race  # unit tests
```

4.3 Running Locally

```
# Start two backend instances on ports 9001 and 9002
./bin/backend -port=9001 &
./bin/backend -port=9002 &

# Start the load balancer
./bin/loadbalancer \
  -port=8080 \
  -algorithm=intelligent \
  -backends=http://localhost:9001,http://localhost:9002

# Run closed-loop load (20 workers, 30s, CPU-only intensity=5)
./bin/client \
  -url=http://localhost:8080 \
  -workers=20 -duration=30s -intensity=5 -mix=cpu:100

# Switch algorithm at runtime
curl -X POST http://localhost:8080/lb/algorithm \
  -H "Content-Type: application/json" \
  -d '{"algorithm":"least_connections"}
```

4.4 Reproducing the Azure Experiments

Requires Azure CLI logged in (`az login`) and an SSH key pair.

```
# Full sweep: 5 workload profiles + recalibrated saturation ramp
./scripts/run-workload-sweep-azure.sh --key-file ~/.ssh/id_rsa

# Saturation ramp only (skips profile sweep)
./scripts/run-workload-sweep-azure.sh \
  --key-file ~/.ssh/id_rsa --saturation-only

# Re-use existing VMs (skip deploy)
./scripts/run-workload-sweep-azure.sh \
  --key-file ~/.ssh/id_rsa --skip-deploy
```

Results are written to `results/$(date)_workload_sweep_azure/` and a human-readable summary to `summary.txt` in that directory. Azure infrastructure is torn down automatically on script exit.

5 Conclusions and Future Work

5.1 Lessons Learned

Least-connections is remarkably effective for free. The single most impactful insight from this project is that tracking in-flight request count — a zero-overhead signal available without any polling — yields consistently superior p99 tail latency on CPU-bound workloads across every profile tested, with a 46–55% p99 reduction over round-robin. It also achieves or exceeds round-robin throughput in all cases. The connection count serves as an implicit capacity signal: a backend that processes requests faster always has fewer active connections, so the algorithm implicitly discovers and exploits compute asymmetry without needing to measure CPU or latency directly.

Network proximity and compute capacity pull routing in opposite directions.

The backend layout creates a fundamental tension: backend-2 (northcentralus) is the closest at ≈ 14 ms RTT but has only 2 vCPUs; backend-3 (eastus2) is farther at ≈ 39 ms but has 4 vCPUs. For I/O-bound workloads, network proximity wins — routing to backend-2 gives the lowest total latency. For CPU-bound workloads, compute capacity wins — routing to backend-3 drains the queue faster despite the extra 25 ms of network overhead. Single-metric algorithms cannot navigate this tradeoff; they pick one dimension and overfit to it.

Lowest-latency is the right tool for the right workload. The lowest-latency algorithm uses each backend’s minimum observed `/metrics` poll RTT (`BaselineRTTs`) as a direct selection metric — pure network-proximity routing. It consistently routes to backend-2 (northcentralus, ≈ 14 ms), the geographically closest node. On I/O-bound workloads, this is *optimal*: because compute time is fixed and identical across backends, minimising network RTT minimises total response time, and lowest-latency achieves the highest throughput of any algorithm on the `io_bound` profile. On CPU-bound workloads, this is *suboptimal*: backend-2 has only 2 vCPUs, so routing *all* CPU-heavy traffic to it (100% in the `cpu_only` profile) causes it to saturate while the 4-vCPU backend-3 sits entirely idle. The saturation point of 75 req/s reflects this over-concentration. The lesson is not that lowest-latency is broken — it is that proximity-only routing is the correct strategy if and only if compute time is not a bottleneck.

Multi-metric scoring spreads load more evenly than single-metric. The intelligent algorithm achieves the best balance: it distributes CPU-only traffic at 24/39/37 (b1/b2/b3), closer to the capacity-ideal 14/29/57 than any single-metric algorithm other than least-connections. It also sustains the highest saturation point among the metric-based algorithms (150 req/s), just one step below the cluster ceiling.

Cold-start and weight recomputation timing matter. The adaptive selector recomputes weights every 10s. Over a 120s experiment window this gives approximately 12 recomputation cycles, enough for the algorithm to converge toward the dominant metric signal rather than being dominated by the cold-start distribution. In earlier experiments

with a 30s recomputation cycle and a 90s window, only 3 recomputations fired per run, and the initial weight shift caused a burst of requests to queue on a newly preferred backend whose metrics had not yet stabilised — this manifested as an apparent saturation artefact at 10req/s. Aligning the recomputation interval to a fraction of the experiment window is critical for adaptive algorithms to produce meaningful results.

Open-loop clients must use context cancellation for safe drain. Early iterations of the load generator used a `select` between a results channel and a `drainDone` channel to unblock in-flight goroutines after the deadline. This has an unavoidable race: when both cases are simultaneously ready, Go selects randomly, meaning goroutines can write to a closed channel. The correct fix is passing a `context.WithCancel` to `http.NewRequestWithContext`: cancelling the context aborts all in-flight HTTP requests, `wg.Wait()` completes deterministically, and `close(results)` is then safe.

5.2 What Remains

- **Weighted round-robin:** Assign static weights proportional to vCPU count. This would route 57% of traffic to backend-3 from the start, requiring no metrics polling and likely matching least-connections on CPU-bound throughput.
- **Request shadowing / canary:** Send a fraction of requests to all backends to maintain live metric estimates even when a backend receives little organic traffic (the starvation problem seen with lowest-latency and backend-2).
- **Adaptive weight smoothing:** The current 10s recomputation cycle eliminates the cold-start artefact, but large variance shifts can cause abrupt routing changes. Adding exponential smoothing ($w_t = \alpha w_{\text{new}} + (1 - \alpha)w_{t-1}$) would prevent oscillation when one metric temporarily spikes.
- **Power-of-two-choices:** Sample two random backends and route to the better one. Expected to combine the simplicity of round-robin with near-optimal load distribution [4].
- **TLS and HTTP/2:** Current implementation is HTTP/1.1 only.

5.3 Connection to CS640 Material

This project directly exercises the core routing concepts covered in CS640. `inteliLB` is a layer-7 routing system: it maintains a global view of backend state, computes a per-request forwarding decision, and updates its policy based on observed network and system conditions — the same separation of concerns as the control plane / data plane architecture studied in lecture.

The load balancer as a routing processor. The CS640 router architecture identifies the *routing processor* as the component that executes routing protocols, maintains routing tables, and computes the forwarding table. `inteliLB`'s main goroutine plays exactly this role at layer 7: it runs the backend health poller (analogous to OSPF `hello` heartbeats), maintains the

backend state table (the link-state database), and computes a forwarding decision for every incoming HTTP request.

ARPANET link-cost metrics and multi-metric scoring. The CS640 lectures trace three generations of ARPANET routing metrics. Metric 1 (hop count) ignores load entirely — the analogue in inteliLB is round-robin. Metric 2 (queue length, i.e. packets enqueued) routes toward the shortest queue regardless of whether that queue leads to the right destination — the analogue is `least_connections`, which the experiments confirm over-concentrates traffic on backend-1. Metric 3 (average delay over a sliding window, $\text{Delay} = (D_T - A_T) + T_{\text{transmit}} + T_{\text{latency}}$) routes by observed end-to-end delay — the analogue is `lowest_latency`. The ARPANET finding that no single metric suffices directly motivates inteliLB’s weighted combination:

$$\text{score}_i = w_L \hat{l}_i + w_C \hat{c}_i + w_K \hat{k}_i$$

where latency, CPU, and connection count are normalised and blended so that over-optimising one dimension does not degrade the others.

Dijkstra and centralized global-state routing. Link-state routing (OSPF) floods topology information to every node so each can run Dijkstra locally over a complete global view. inteliLB follows the same model at the application layer: the load balancer polls every backend every 5 s to maintain a complete, current view of the system, then scores all backends in $O(n)$ to pick the best next hop for each request. The centralized, globally-informed decision is what distinguishes `intelligent` and `adaptive` from the reactive, locally-scoped heuristics (`lowest_latency`, `lowest_cpu`) that consider only one dimension at a time.

Convergence, cold start, and transient disruption. The CS640 treatment of link-state convergence identifies two failure modes: (1) a detection-delay black hole, where a node continues forwarding to a failed neighbor until heartbeat timeouts expire; and (2) inconsistent link-state, where routers that have not yet received a flooded update make contradictory forwarding decisions. Both appear in inteliLB. The cold-start window — before any backend has been polled — is the detection-delay analogue: metric-based algorithms have no cost information and fall back to round-robin to avoid routing into an unknown-state backend. The 5-second poll interval is the convergence window: during it, the load balancer’s view of backend CPU and latency may be stale, causing transient mis-routing analogous to the inconsistent link-state window during OSPF flooding.

Link-state vs. distance-vector trade-off. CS640 contrasts link-state (high messaging overhead, fast convergence) with distance-vector (low overhead, slow convergence and count-to-infinity risk). inteliLB’s polling architecture is a link-state model: the single load balancer polls all backends directly, incurring $O(n)$ messages per interval but converging in one round. A gossip-based, peer-to-peer health model would be the distance-vector analogue — lower polling overhead at the cost of slower, potentially inconsistent convergence. The current single-LB design is also structurally vulnerable to the inconsistent-state pathology when scaled to multiple LB instances without coordination, exactly as un-synchronized routers produce transient forwarding loops during topology changes.

Adaptive recomputation and periodic flooding. OSPF floods link-state packets both on topology events (link failure) and periodically (every ~ 30 minutes) to ensure consistency. `inteliLB`'s adaptive algorithm mirrors this dual trigger: the 5-second poller is the event-driven channel (detects backend CPU and latency shifts), while the 30-second weight recomputation is the periodic recalculation that adjusts the routing policy even when no discrete event has occurred. The 30-second interval proved too coarse in the experiments — the adaptive algorithm was frequently caught mid-adjustment during the 120-second test windows, producing erratic distributions (0% to one backend in the realistic profile), consistent with the transient disruptions that occur when periodic flooding intervals are too long relative to the rate of topology change.

Saturation and queueing theory. The saturation ramp results (Figure 6) exhibit the classic hockey-stick latency profile predicted by $M/M/k$ queueing theory: latency remains near the service time until utilisation approaches 100%, then diverges. `intelligent` sustains clean throughput to 150req/s before saturation while `round_robin` and `lowest_latency` collapse earlier, because smarter routing keeps all k servers below their individual saturation points longer — directly applying the CS640 insight that load distribution determines effective system capacity.

References

- [1] NGINX Inc. *NGINX Load Balancing*. 2024. <https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/>
- [2] HAProxy Technologies. *HAProxy Configuration Manual*. 2024. <https://www.haproxy.com/documentation/>
- [3] D. Karger et al. *Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web*. STOC 1997.
- [4] M. Mitzenmacher. *The Power of Two Choices in Randomized Load Balancing*. IEEE Transactions on Parallel and Distributed Systems, 2001.
- [5] Y. Lu, Q. Xie, G. Klier, A. Geller, J. Larus, A. Greenberg. *Join-Idle-Queue: A Novel Load Balancing Algorithm for Dynamically Scalable Web Services*. Performance Evaluation, 2011.
- [6] D. Eisenbud et al. *Maglev: A Fast and Reliable Software Network Load Balancer*. NSDI 2016.
- [7] A. Donovan, B. Kernighan. *The Go Programming Language*. Addison-Wesley, 2015.